

ONNX → HEF Conversion Guide for Hailo-10H (Raspberry Pi 5 + Hailo HAT+)

This guide walks through compiling an ONNX model into a `.hef` binary that runs on the **Hailo-10H** accelerator (Raspberry Pi 5 + Hailo HAT+).

The flow is:

1. Capture a **real calibration dataset** on the Pi (the camera + scene you will deploy with).
2. Copy that `.npy` to the workstation that has the **Hailo Dataflow Compiler 5.3.0** installed.
3. Run `convert_hef_file.py` to translate → quantize → compile to HEF.

Target device: `hailo10h`. Do **not** use `hailo8` or `hailo8l` — the resulting HEF will not load on the Hailo-10H runtime.

0. Prerequisites

Workstation (compilation host)

- Conda environment
- Hailo Dataflow Compiler installed
- Pinned dependencies from `week-4/hailo_constraints.txt` :

```
numpy==1.26.4
scipy==1.12.0
protobuf==3.20.3
onnx==1.17.0
onnxruntime==1.18.0
matplotlib==3.5.2
opencv-python<4.10
ultralytics==8.4.52
```

Activate the environment before any command below:

```
conda activate exporthyolov26n
# sanity check
python -c "import hailo_sdk_client, sys; print('hailo_sdk_client OK')"
```

Raspberry Pi 5 + Hailo-10H HAT+

- picamera2 available (`sudo apt install python3-picamera2`)
- The same camera + lighting + framing you will use at inference time

1. Capture a real calibration dataset (on the Pi)

Random calibration data **kills INT8 accuracy** (class logits collapse).

Capture ~128 real frames from the deployment camera with
`capture_calib_dataset.py` .

```
# On the Raspberry Pi 5, in the project folder
cd week-4
```

```
# Default: 128 frames at 640x640 from the Pi Camera -> calib_real.npy
python3 capture_calib_dataset.py \
    --num-frames 128 \
    --imgsz 640 \
    --output calib_real.npy \
    --delay 0.1
```

While it runs, **slowly move the camera through the scene** so the set covers the brightness / texture range you expect at inference.

Other useful variants:

```
# Larger / higher-res calibration set
python3 capture_calib_dataset.py --num-frames 256 --imgsz 640 \
    --output calib_scene.npy --delay 0.2

# No camera handy? Use a video file captured earlier from the same scene
python3 capture_calib_dataset.py --source video --input clip.mp4 \
    --num-frames 128 --output calib_clip.npy
```

The output is an **NHWC float32 array in [0, 1]** (shape (N, imgsz, imgsz, 3)) — exactly what `convert_hef_file.py --calib-npy` expects.

Copy the file to the workstation:

```
# From the workstation
scp raspkit00@<raspberrypi-ip>:~/path/to/week-4/calib_real.npy week-4/
```

2. Convert ONNX → HEF for Hailo-10H (on the workstation)

2.1 Generic model (e.g. MobileNetV2, classifier)

```
conda activate exportyolov26n
cd week-4

python3 convert_hef_file.py \
    --onnx mobilenetv2_cifar10.onnx \
    --hef mobilenetv2_cifar10.hef \
    --hw-arch hailo10h \
    --calib-npy calib_real.npy
```

2.2 YOLO26 (recommended path for this lab)

YOLO26's detection head contains ops (TopK, GatherElements, Mod) that the Hailo parser cannot lower. Cut the graph at the head transpose and let post-processing run on the CPU:

```
python3 convert_hef_file.py \
  --onnx yolo26n.sim.onnx \
  --hef yolo26n_calibrated.hef \
  --hw-arch hailo10h \
  --calib-npy calib_real.npy \
  --end-node /model.23/Transpose
```

2.3 Without a real calibration set (FPS-only benchmarking)

Random data is **valid only for measuring throughput**, not accuracy:

```
python3 convert_hef_file.py \
  --onnx yolo26n.sim.onnx \
  --hw-arch hailo10h \
  --calib-shape 100 640 640 3 \
  --end-node /model.23/Transpose
```

2.4 Useful flags

Flag	Purpose
--hw-arch hailo10h	Required for Hailo-10H. Default in the script.
--calib-npy <path>	Use the real NHWC float32 calibration set from step 1.
--calib-shape N H W C	Shape of random calibration data (ignored if --calib-npy).
--end-node <name>	Cut the ONNX graph here (drop unsupported head ops). Repeatable.
--start-node <name>	Override graph input(s). Rarely needed.
--model-name <name>	Internal model name (default: ONNX file stem).
--gpu	Let Hailo's internal TF use the GPU. Off by default — the CPU path is more reliable in conda envs with mismatched CUDA/cuDNN.
--seed <int>	Reproducible random calibration data.

2.5 What the script does

1. Forces TF onto the CPU (CUDA_VISIBLE_DEVICES=-1) unless --gpu .

2. `ClientRunner(hw_arch="hailo10h")` .
3. `runner.translate_onnx_model(...)` — optionally with `end_node_names` .
4. `runner.optimize(calib_dataset)` — INT8 quantization using your calib set.
5. `runner.compile()` → writes the `.hef` file.

3. Verify the HEF

```
# Quick header sanity check (architecture string should mention hailo10h)
hailortcli parse-hef yolo26n_calibrated.hef | head
```

Then copy the HEF to the Pi and run it with your inference script
(e.g. `pipeline_template.py` , `infer_single.py` , or `benchmark.py`).

```
scp yolo26n_calibrated.hef pi@<raspberrypi-ip>:~/path/to/week-4/
```